Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Bu derste önce R basamaklı palindrom sayıların sayısı konusu netleştirilir. Ardından interpolasyon araması (interpolation search) kodlanmaya çalışılır, Dev-C++'ta hata ayıklama (debug) gösterilir ve arama algoritmaları karşılaştırılır. Sonra iki sıralı diziyi birleştirme (merge) algoritması akış diyagramı ve C koduyla yazılır. Kaydın sonunda birinci laboratuvar sınavının sorusu anlatılır.
- Laboratuvar sonrası notlar
  - Laboratuvar kodlarını asistanlar değerlendirir. Belli bir puanlama cetveli vardır. Puanlara itiraz notlar açıklandıktan sonra ilgili asistana mail ile yapılır.
  - Kod yazarken sık sık derleyin. Hatayı en sonda aramak yerine her adımda kontrol etmek, hatayı daha küçük bir bölgede aramak demektir. Bu da ikili aramaya benzer.
  - Laboratuvar ve sınavlarla ilgili eleştiriler Telegram veya mail yerine derste sorulmalıdır. Aynı soruların farklı kanallarda tekrar tekrar cevaplanması zaman kaybıdır.
- R basamaklı palindromlar: yazdırmak ile saymak
  - Sayıları yazdırmak için, R'ye göre değişen sayıda iç içe döngü yazılamaz. Genel çözüm: 10^{R-1}'den $10^R - 1$'e tek bir döngüyle her sayıyı ters çevirip kendisiyle karşılaştırmak. Basamak sayısı biliniyorsa (ör. 6) ABCCBA üretimi çok daha hızlıdır (900.000 yerine 900 tur).
  - Saymak için formül kullanılır. Dıştaki basamak çiftine 9 farklı değer (1-9), içteki her basamak çiftine 10 farklı değer gelebilir: 1 ve 2 basamakta 9, 3 ve 4 basamakta 90, 5 ve 6 basamakta 900. Genel formül: $9 \times 10^{\lfloor (R-1)/2 \rfloor}$.
  - Formülde bir üs alma ve bir çarpma vardır. Üs, bir döngüyle yaklaşık $R/2$ adımda hesaplanır. Bütün sayıları tek tek kontrol etmek ise 10^R mertebesinde tur gerektirir.
- Ödev sorusuna bir öğrenci önerisi
  - Laboratuvar sorusunda (ardışık dizide eksik sayı) bir öğrenci, beklenen toplam ile girilen sayıların toplamının farkından eksik sayıyı bulmayı önerdi. Hoca, ikili aramanın logaritmik olduğunu, bu yaklaşımın ise "sabit" bir algoritmaya benzediğini ve doğru sonuç veriyorsa daha iyi sayılabileceğini söyler. Öneriyi asistanlara iletir.
- İkili aramayı `do-while` ile yazmak
  - Bir öğrenci ikili aramanın `do-while` ile yazılıp yazılamayacağını sordu. Böylece `mid` hesabı döngüden önce ve içinde iki kez yazılmaz.
  - İlk denemede kod yanlış sonuç verdi (14'ü bulamadı, 88'i "buldu"). Hoca kodu eski hâline döndürdü. Sonra `do { mid = (low + high) / 2; ... } while (low <= high && a[mid] != key);` yapısını yeniden kurdu ve 13'ü ilk denemede buldu.
  - `low = mid + 1` ve `high = mid - 1` yazılır, çünkü `mid`'deki elemana zaten bakılmıştır. Aynı elemana tekrar bakmamak için bir sağından veya bir solundan devam edilir.
- İnterpolasyon araması
  - İkili arama, aralığı her seferinde $\frac{1}{2}$ katsayısıyla böler (bisection yöntemi gibi). İnterpolasyon araması bu katsayıyı, aranan değerin aralıktaki konumuna göre seçer. Bu, Sayısal Analiz'deki regula falsi (doğrusal interpolasyon) yönteminin karşılığıdır. Hoca bunu "rastgele ortaya ateş etmek yerine dürbünle nişan almak" olarak anlatır.
  - Derste formül önce regula falsi formülünden ($c = \frac{bf(a) - af(b)}{f(a) - f(b)}$) uyarlanarak yazıldı, ama kod sonsuz döngüye girdi veya elemanları bulamadı. Debug ile değişkenler incelendi. Sonra TutorialsPoint'teki örnek kodla karşılaştırıldı: `mid = lo + ((double)(hi - lo) / (list[hi] - list[lo])) * (data - list[lo]);`.

  - Örnek kodda `break` vardı. Hoca `break` kullanmadan, bulununca doğrudan `mid` döndürülecek şekilde düzenledi ve ilk tahminde bulunan eleman için döngüye hiç girilmediğini gösterdi.
  - Hoca formülü tekrar inceleyeceğini ve kodu paylaşacağını söyledi.
- Dev-C++'ta hata ayıklama (debug)
  - Debug için kod tek bir dosya olarak değil, bir proje olarak açılmalıdır (File > New Project, içine yeni bir kaynak dosya). Derleyici ayarında debug modu seçili olmalıdır.
  - Satır numarasına tıklanınca satır kırmızı olur. Bu bir durma noktasıdır (breakpoint). Execute > Debug ile program bu noktaya kadar çalışır.
  - "Add watch" ile bir değişkenin veya ifadenin değeri izlenir. Dizinin adı izlenirse adresi görünür.
  - "Next line" bir sonraki satıra geçer. "Into function" çağrılan kendi fonksiyonumuzun içine girer (`printf` gibi kütüphane fonksiyonlarına girilmez). "Skip function" fonksiyonun içine girmeden geçer. "Next instruction" döngünün içinde adım adım ilerler.
  - Debug, özellikle yanlış sonuç veren mantıksal hataları bulmakta çok faydalıdır.
- Arama algoritmalarının karşılaştırılması

Algoritma	Ön koşul	En iyi durum	En kötü durum
Sıralı arama (sequential)	yok	1	n
İkili arama (binary)	dizi sıralı	1	$\log_2 n$
İnterpolasyon araması	dizi sıralı ve değerler düzgün dağılmış	1	$\log_2 \log_2 n$ (hocanın verdiği değer, ders sonrası nota bakın)

- Karmaşıklıkların açıklaması
  - Sıralı arama ilk elemandan başlar. Şanslıysak ilk elemanda buluruz. Eleman yoksa veya en sondaysa n karşılaştırma yapılır.
  - İkili arama ilk olarak ortadaki elemana bakar. Her adımda arama uzayı yarıya iner: n, $n/2$, $n/4$, ..., $n/2^i$. $n/2^i = 1$ olunca $i = \log_2 n$. 1024 elemanlık bir dizide en fazla yaklaşık 10 adımda sonuca ulaşılır.
  - İnterpolasyon aramasında 2^{16} elemanlık dizi için $\log_2 \log_2 2^{16} = \log_2 16 = 4$ adım yeterlidir. İkili aramada 16 adım gerekir.
  - Hoca karesel (kuadratik) interpolasyonun da kullanılabileceğini ekler. Sayısal Analiz'de interpolasyon konusunda işlenecektir.
- İki sıralı diziyi birleştirme (merge)
  - 1. yaklaşım (bir öğrencinin önerisi): iki diziyi art arda üçüncü diziye kopyala ve sırala. Kopyalama $2n$, bubble sort n^2 adım. Toplam yaklaşık $n^2 + 2n$.
  - 2. yaklaşım: her dizi için bir indis tut (`i`, `j`) ve sonuç dizisi için `k`. `a[i]` ile `b[j]`'den küçük olanı `c[k]`'ya kopyala ve onun indisini artır.
  - Bu yaklaşımı `for (k = 0; k < m + n; k++)` ile yazmak, dizilerden biri önce bittiğinde dizinin sınırı dışına erişime yol açar. Örneğin `a` = 1..6, `b` = 7..12 ise `a` bittikten sonra `a[6]` ile karşılaştırma yapılmaya çalışılır.
  - Bunun bir çözümü, iki dizinin sonuna "sınır bekçisi" olarak `INT_MAX` (`limits.h`) koymaktır. Böylece biten dizinin son değeri her zaman büyük kalır. Ama bu bellek harcar.
  - Daha iyi çözüm: `while (i < m && j < n)` ile iki dizi de bitmemişken karşılaştır. Biri bitince döngüden çıkılır. Diğer dizide kalan elemanlar zaten sıralıdır ve karşılaştırma yapılmadan doğrudan sona kopyalanır (`while (i < m)` ve `while (j < n)`). Bu iki döngüden yalnızca biri çalışır.
  - Bir öğrencinin belirttiği gibi `k` her zaman `i + j`'ye eşittir.
  - Analiz: iki dizi de n elemanlıysa, elemanlar hangi sırayla gelirse gelsin toplam adım sayısı $2n$'dir (genel olarak $m + n$). Örnekler: (1, 3, 5, 7 / 2, 4, 6, 8) dönüşümlü gider, (1, 2, 3, 4 / 5, 6, 7, 8) birinci dizi önce biter, karışık bir durumda ana döngü $2n - 1$ kez, kalan döngü 1 kez döner. n^2'lik maliyetten $2n$'ye inilmiş olur.
  - Merge algoritması, ileride Veri Yapıları ve Algoritma Analizi derslerinde görülecek merge sort'un temelidir.
- Merge'in C kodu
  - Diziler `#define MAX 10` ile tanımlanır. Elemanlar rastgele sayılarla doldurulur (`time.h`

ve rastgele sayı üretimi bir sonraki derste anlatılacaktır), sonra `bubble_sirala` ile sıralanır.
  - `dizi_yazdir` fonksiyonuna üçüncü bir parametre (`char ch[]`) eklenerek "before sort" / "after sort" gibi bir başlık da yazdırılır. Yeni bir argüman eklenince tanıma da aynı tipte bir parametre eklenmelidir.
  - `merge(a, MAX, b, MAX, c)` çağrılır (yazdırma `dizi_yazdir(c, 2*MAX, ...)` ile yapılır). `c` dizisi iki dizinin toplamı kadar yer tutmalıdır. `dizi_yazdir`'ın `MAX`'a değil kendisine gelen `size` parametresine göre çalışması gerekir. Aksi hâlde birleştirilmiş dizinin yalnızca ilk 10 elemanı yazdırılır.
  - Fonksiyon `int` döndürecek şekilde değiştirilerek `k` döndürüldü. 10 + 10 elemanlı dizilerde, değerler değiştirilse de adım sayısı her seferinde 20 çıktı.
  - Değer verilmemiş dizilerde rastgele "çöp" değerler görülebilir. C belleği sıfırlamaz.
- Değişkenlerin tanımlanma yeri
  - Bir öğrencinin sorusu üzerine: değişkenler fonksiyonun başında, işlemlerden önce tanımlanmalıdır. `for (int i = 0; ...)` C99'a özgüdür ve derste kullanılan derleyicide hata verir. Hoca `srand` için "makro" dedi; doğrusu `srand` bir kütüphane fonksiyonudur ve o satır bir işlemdir. C89 kuralına göre tanımlar bu çağrıdan önce gelmelidir. Ders sonrası denemede `-std=c89 -pedantic` "ISO C90 forbids mixed declarations and code" uyarısı verdi, `-std=gnu89` vermedi.
- Birinci laboratuvar sınavı (YAPISAL PROGRAMLAMAYA GİRİŞ LAB-1)
  - Soru: Ardışık sayılardan oluşan sıralı bir dizide, ardışıklığı bozan kayıp sayıyı bulan algoritmayı kodlamak.
  - Girdi: dizinin boyutu ve elemanları (kullanıcıdan). Çıktı: kayıp sayı varsa sayının kendisi, yoksa "No missing value".
  - Örnekler: 2 3 4 5 7 8 9 10 için 6. −8 −7 −6 −5 −3 −2 −1 0 için −4. 10 11 12 için "No missing value".
  - Önemli: program büyük boyutlu diziler düşünülerek verimli tasarlanmalıdır. İpucu: sayılar sıralı olduğu için sıralı dizide aramayı kolaylaştıran yöntemler kullanılmalıdır. Diziyi baştan sona tek tek kontrol etmek beklenmez.
  - Diziler her zaman birer birer artar, azalmaz. Negatif sayılardan veya ortadaki bir sayıdan başlayabilir. Birden fazla kayıp sayı varsa algoritmanın ilk bulduğu sayıyı yazdırmak yeterlidir.
  - Boyut `#define MAX` ile sabitlenebilir. Test için küçük diziler girilebilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Kodu derleye derleye ilerleyin. Hata en sonda bulunursa düzeltmek için zaman kalmayabilir.
- Algoritmada yalnızca verimlilik değil, doğruluk da önemlidir. Daha hızlı ama yanlış sonuç veren bir algoritma kabul edilmez.
- `break` kullanmadan da döngüden çıkılabilir. `break`'li çözüm hem ekstra değişken hem de gereksiz bir dolambaç içerir. Hoca bunu "sağ kulağı sağ elle değil, başın arkasından dolaşarak sol elle tutmak" olarak anlatır.
- İkili arama ve interpolasyon araması yalnızca sıralı dizilerde çalışır.
- Merge algoritması önemlidir ve birçok yerde karşınıza çıkacaktır.
- Hoca kodları doğrudan değil ekran görüntüsü olarak paylaşır. Kodu yeniden yazmak, hata yapmak ve düzeltmek öğrenmeyi sağlar.
- Laboratuvar için: aklınıza gelen ilk yöntem iyi bir yöntem olmayabilir. Derslerde konuşulan "iyi algoritma" ölçütlerini dikkate alın.

Kısa Tekrar Notları

- R basamaklı palindrom sayısı: $9 \times 10^{\lfloor (R-1)/2 \rfloor}$.
- İkili arama: `low = mid + 1` / `high = mid - 1`, en kötü $\log_2 n$.
- İnterpolasyon: `mid = low + (key - a[low]) * (high - low) / (a[high] - a[low])`. Sıralı ve düzgün dağılmış veri.
- Merge: `while (i < m && j < n)` karşılaştırarak kopyala, sonra kalanları kopyala. Maliyet $m + n$.
- Debug: proje aç, breakpoint koy, Add watch, Next line / Into function / Skip function.

Detaylı Açıklamalar

Merge fonksiyonu

Derste yazılan fonksiyonun ekran görüntüsünden derlenmiş hâli. Ekranda ikinci döngünün şartı önce `j<m` yazılmıştı. Bir öğrencinin uyarısıyla `j<n` olarak düzeltildi.

```
int merge(int a[], int m, int b[], int n, int c[])
{
    int i = 0, j = 0, k = 0;
    while (i < m && j < n) {
        if (a[i] <= b[j]) {
            c[k] = a[i++];
        }
        else {
            c[k] = b[j++];
        }
        k++;
    }
    while (i < m) {
        c[k++] = a[i++];
    }
    while (j < n) {
        c[k++] = b[j++];
    }
    return k;
}
```

- `c[k] = a[i++];` önce `a[i]`'yi atar, sonra `i`'yi artırır.
- `a[i] <= b[j]` yazılarak eşitlik durumu ayrı bir `if` gerektirmeden ele alınır.
- Hoca ilk başta `c`'nin boyutu için `int l` parametresi de yazmıştı. Bir öğrenci kullanılmadığını söyleyince sildi (01:25:01-01:25:56), çünkü `k`'nin son değeri zaten `m + n`'dir. Son imza beş parametrelidir.

Ders sonrasında kod `gcc -Wall -std=c89 -pedantic` ile derlendi. 2 4 6 8 10 ve 1 3 5 7 9 dizileri 1 2 3 4 5 6 7 8 9 10 olarak birleştirildi ve fonksiyon 10 döndürdü.

Laboratuvar sorusu için bir çözüm (derste yazılmadı)

Aşağıdaki çözüm derste yazılmamıştır. Laboratuvar ipucuna uygun olarak ikili aramayla yazılmış ve ders sonrasında test edilmiştir. Fikir şudur: ardışıklık bozulmamışsa her `i` için `a[i] - a[0] == i` olur. İlk bozulduğu indis ikili aramayla bulunur ve kayıp sayı `a[0] + low`'dur. Döngüde `break` kullanılmaz, bütün kod `main` içindedir.

```
#include <stdio.h>
#define MAX 100

int main()
{
    int a[MAX], n, i, low, high, mid;

    printf("Eleman sayisi: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    if (a[n-1] - a[0] == n - 1) {
        printf("No missing value\n");
    } else {
        low = 0;
        high = n - 1;
        while (low < high) {
```

```
            mid = (low + high) / 2;
            if (a[mid] - a[0] == mid)   /* buraya kadar ardisiklik bozulmamis */
                low = mid + 1;
            else
                high = mid;
        }
        printf("%d\n", a[0] + low);
    }
    return 0;
}
```

`gcc -Wall -std=c89 -pedantic` ile derlendi. Sorudaki üç örnek için 6, −4 ve "No missing value" verdi. Ayrıca −2 −1 1 2 için 0, 1 3 4 5 6 için 2, 1 2 3 4 6 için 5 verdi. İşlem sayısı $\log_2 n$ mertebesindedir.

Ders sonrası notlar

- **İnterpolasyon formülündeki hata:** Derste regula falsi formülü $c = \frac{bf(a)-af(b)}{f(a)-f(b)}$ doğrudan dizi değerleriyle (`f(a) = a[low]`) yazıldı ve kod çalışmadı. Regula falsi bir fonksiyonun sıfırını bulur. Dizide `key` aranırken fonksiyon $f(i) = a[i] - key$ olmalıdır. Yani formülde `a[low]` yerine `a[low] - key`, `a[high]` yerine `a[high] - key` kullanılmalıdır. Bu yerine koyma yapılınca formül, TutorialsPoint'teki koddaki hâline dönüşür: $mid = low + \frac{(key-a[low])(high-low)}{a[high]-a[low]}$. Ders sonrasında bu formülle yazılan sürüm, dersteki dizide 14'ü buldu (indis 5) ve 88 için -1 döndürdü. Sağlam bir sürümde ayrıca `key`'in `a[low]` ile `a[high]` arasında olduğu kontrol edilmeli ve `a[high] == a[low]` durumunda sıfıra bölme önlenmelidir.
- **"Uniform distribution" ne demek:** Derste düzgün dağılım (uniform distribution) "her elemanın frekansının eşit olması" olarak tanımlandı. İnterpolasyon araması için gereken şey, değerlerin aralık boyunca eşit aralıklarla, yani doğrusala yakın dağılmasıdır (ör. 10, 20, 30, ... veya 8, 11, 13, 16, ...). Değerlerin her birinden kaç tane olduğu değil, değerlerin aralığa nasıl yayıldığı önemlidir. Bu durumda formül aranan değerin yerini iyi tahmin eder.
- **İnterpolasyon aramasının karmaşıklığı:** $\log \log n$, düzgün dağılmış veride ortalama durum karmaşıklığıdır. En kötü durumda (ör. değerler üstel artıyorsa) interpolasyon araması n adıma kadar çıkabilir. Bu yüzden veri dağılımı bilinmiyorsa ikili arama daha güvenlidir.
- **İkili aramada en kötü durum:** Tam olarak $\lfloor \log_2 n \rfloor + 1$ karşılaştırmadır. Hoca da "artı 1"in algoritma analizi dersinde ele alınacağını söyledi.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.